DevOps Homework

We would like to see how you approach projects. It does not matter if you are already familiar with any of the mentioned technologies or not, we just want to see how you tackle a problem and come up with a solution. The complexity is up to you, but you can also prepare a minimum viable setup and suggest further improvements which could take place down the line..

Your objective is to write a plan a.k.a specification (or spec) for a simple production service, where each change of the service goes through a CI/CD (continuous integration/continuous delivery) pipeline and is automatically deployed to production. Coding and building the service and pipeline is optional. When writing the plan please describe how you would approach the problem, steps that you would do, and add any documentation links you think are relevant at different steps and why.

What is used at Celtra

- Cloud computing – Amazon Web Services (AWS) (other alternatives: Google Cloud, Heroku, DigitalOcean)
- Infrastructure as Code (IaC) - AWS CDK (Cloud Development Kit preferably in Typescript) and HashiCorp Terraform
- Observability - Cloudwatch, Grafana Cloud (other alternatives: DataDog, PostHog,...)
- CI/CD - GitHub Actions (other alternatives: GitLab Pipelines, Jenkins, Codefresh, GoCD, …)
- Languages: PHP, Scala, Python, Go,…

Project instructions

1. Create an HTTP service

Prepare a simple HTTP service that returns dynamic data. It should expose one GET route. A route should be available on **<accountId>/data.** Response on this endpoint should look like this:

```
{
     "accountId": "<accountId>",
     "timestamp": <timestamp in UTC>,
     "data": "<some random string>"
}
```

Include tests to ensure the service works correctly and include running those tests in the integration pipeline before pushing any new changes to production.

2. Create a Docker image

Complete service should be enclosed in a single Docker image. It does not need any other external dependencies or additional persistent storage.

3. CI step

CI tool of choice should build the container image and run tests. CI should be done in multiple stages, meaning the build step should be separated from the test run step.

4. CD step

After CI has passed all tests, CD pipeline should deploy the service to production.

Go through each of the above steps and check freely available documentation (or consult your favourite LLM) to familiarize yourself with them. All of them can either have a lot of intricate details or be very simple. If you submit any code for this project, it needs to be readable and understandable, so add comments where needed. No matter how you present your solution it is important that at the end of your project you have a high level overview of what needs to be done.